Producto 06 — Procedimiento de implementación

Servicio Nacional de Aprendizaje (SENA)
Programa: Análisis y Desarrollo de Software
Ficha: 3145555
Proyecto: SmartHome
Elaborado por: Natalia Chala Chala, Kevin Stiven López Amaya y Karen Daniela Holguín Cruz
Instructor: José de Jesús Motta Vargas
Fecha de actualización: 6 de noviembre de 2026

1. Presentación

Este documento organiza el procedimiento de implementación de SmartHome para entorno local y de validación académica. Se construye a partir de los repositorios reales del proyecto y toma como referencia la documentación histórica del equipo, pero corrige el problema más importante detectado en versiones anteriores: el uso de comandos, rutas y configuraciones genéricas sin verificar.

2. Objetivo y alcance

El propósito es dejar una ruta ordenada para levantar los componentes principales del proyecto:

• Base de datos
• Backend
• Frontend web
• Frontend móvil

Cuando se menciona hardware o integración IoT, se hace como procedimiento complementario y condicionado a que existan los recursos de laboratorio disponibles. Este documento no incluye secretos, contraseñas reales, IP privadas ni valores productivos.

3. Repositorios que intervienen en la implementación

Componente	Repositorio
Base de datos	https://github.com/nataliachala1/smarthome-database
Backend	https://github.com/nataliachala1/smarthome-backend
Frontend web	https://github.com/nataliachala1/smarthome-frontend-web
Frontend móvil	https://github.com/Kevin-Stiven-Lopez-Amaya404/from_movil
Documentación (referencia)
https://github.com/Kevin-Stiven-Lopez-Amaya404/smart-home-docs |

4. Requisitos previos del entorno

4.1 Software recomendado

• Git
• Node.js y npm (versiones compatibles con cada repositorio)
• PostgreSQL
• Liquibase o el contenedor/procedimiento definido por el repositorio de base de datos
• Docker y Docker Compose (si el repositorio o el entorno lo requieren)
• Navegador web moderno
• Android Studio o Expo Go / build de desarrollo para el cliente móvil

4.2 Buenas prácticas antes de iniciar

1. Trabajar siempre desde un directorio de proyecto ordenado.
2. Clonar cada repositorio por separado.
3. Revisar el archivo README.md y .env.example de cada componente antes de crear variables.
4. No inventar nombres de variables si el proyecto ya trae plantillas oficiales.

5. Estructura recomendada de carpetas

```
SmartHome/
??? smarthome-database/
??? smarthome-backend/
??? smarthome-frontend-web/
??? from_movil/
```

6. Paso a paso de implementación

6.1 Clonar los repositorios

```
git clone https://github.com/nataliachala1/smarthome-database.git
git clone https://github.com/nataliachala1/smarthome-backend.git
git clone https://github.com/nataliachala1/smarthome-frontend-web.git
git clone https://github.com/Kevin-Stiven-Lopez-Amaya404/from_movil.git
```

6.2 Preparar la base de datos

Fuente de verdad: el repositorio smarthome-database.

Procedimiento recomendado:

1. Entrar al repositorio de base de datos.
2. Identificar el README, docker-compose, changelog-master.yaml, scripts SQL o archivos de propiedades disponibles.
3. Configurar las credenciales de entorno en archivos locales no versionados.
4. Ejecutar validate antes de aplicar cambios.
5. Ejecutar las migraciones según el procedimiento definido por ese repositorio.

Nota importante: no se debe asumir que npx liquibase update desde el backend sea siempre la ruta oficial. Si el repositorio de base de datos define otro flujo, ese flujo prevalece.

6.3 Configurar el backend

1. Ingresar al repositorio smarthome-backend.
2. Instalar dependencias.
3. Crear el archivo .env a partir de .env.example o del inventario oficial de variables.
4. Configurar la conexión a la base de datos ya preparada.
5. Configurar valores de autenticación, puertos y cualquier variable de integración documentada.
6. Ejecutar el proyecto en modo desarrollo y confirmar que levante correctamente.

Ejemplo general:

```
cd smarthome-backend
npm install
```

```
crear .env a partir de .env.example
npm run start:dev
```

Variables típicas a validar (solo como categorías, no como valores definitivos):

• DATABASE_URL
• PORT
• JWT_* o variables de autenticación equivalentes
• variables de correo si el proyecto las usa
• variables MQTT o de tiempo real, si están definidas

6.4 Configurar el frontend web

1. Ingresar al repositorio smarthome-frontend-web.
2. Instalar dependencias.
3. Crear el archivo .env o equivalente tomando como base la plantilla oficial.
4. Definir la URL del backend sin usar secretos.
5. Ejecutar el servidor de desarrollo.

Ejemplo general:

```
cd smarthome-frontend-web
npm install
npm run dev
```

La variable principal suele ser la URL base de la API. Debe apuntar al backend levantado en el paso anterior.

6.5 Configurar el frontend móvil

1. Ingresar al repositorio from_movil.
2. Instalar dependencias.
3. Revisar el archivo de entorno o configuración pública de Expo.
4. Ajustar la URL de la API para que el celular o emulador alcance el backend.
5. Ejecutar Expo o la build de desarrollo correspondiente.

Ejemplo general:

```
cd from_movil
npm install
npx expo start
```

Si el entorno usa variables públicas de Expo, debe respetarse la convención existente en el repositorio (por ejemplo variables EXPO_PUBLIC_*).

7. Verificaciones mínimas posteriores al levantamiento

Verificación	Resultado esperado
Base de datos disponible	El backend puede conectarse sin errores de conexión.
Backend activo	Responde en el puerto configurado y registra arranque correcto.
Frontend web activo	La aplicación carga y consume la API configurada.
Frontend móvil activo	La app inicia y puede apuntar al backend del entorno de prueba.
Migraciones consistentes	El esquema creado coincide con el repositorio de base de datos.

8. Integración IoT y hardware (condicionada)

La documentación histórica menciona dispositivos compatibles, MQTT y trabajo con Shelly 1PM Gen4. Ese procedimiento solo debe incluirse como definitivo si el equipo dispone de un entorno real de validación.

Recomendaciones editoriales para este punto

• Si no hubo práctica real con hardware, redactar esta sección como procedimiento propuesto.
• Si sí hubo práctica real, anexar evidencia: fotos, bitácora, logs o capturas.
• No fijar IP privadas ni credenciales del laboratorio en el documento final.

9. Problemas frecuentes y acciones sugeridas

Problema	Posible causa	Acción sugerida
El backend no conecta a PostgreSQL	DATABASE_URL incorrecta o BD no levantada	Revisar variables y estado de PostgreSQL
El frontend no consume la API	URL base incorrecta o CORS	Validar variable de API y configuración del backend
La app móvil no alcanza el backend	IP local o red incorrecta	Verificar que el celular y el servidor estén en la misma red
Migraciones fallan	Configuración Liquibase incompleta	Revisar propiedades y ejecutar validate antes de update
Integración IoT no responde	entorno MQTT o hardware no preparado	Separar diagnóstico de software y laboratorio

10. Checklist final de implementación

• [ ] Repositorios clonados correctamente.
• [ ] Base de datos preparada desde el repositorio oficial.
• [ ] Backend levantado con variables locales correctas.
• [ ] Frontend web funcionando contra el backend.
• [ ] Frontend móvil apuntando al backend de pruebas.
• [ ] Variables sensibles excluidas del documento y del control de versiones.
• [ ] Evidencia mínima de levantamiento recolectada.

11. Conclusión

La presente versión del procedimiento de implementación mejora la confiabilidad del producto 06 porque se apoya en los repositorios reales del proyecto y en la separación correcta de responsabilidades entre base de datos, backend y clientes. Además, evita copiar comandos desactualizados o secretos de ejemplo y deja claramente diferenciados los pasos verificados, los dependientes del entorno y los condicionados al laboratorio de hardware.